









GDG Pescara





DevFest
Pescara 2026



Dalla Sicofanzia alla Realtà: Agent Harness Deterministici per l'SDLC

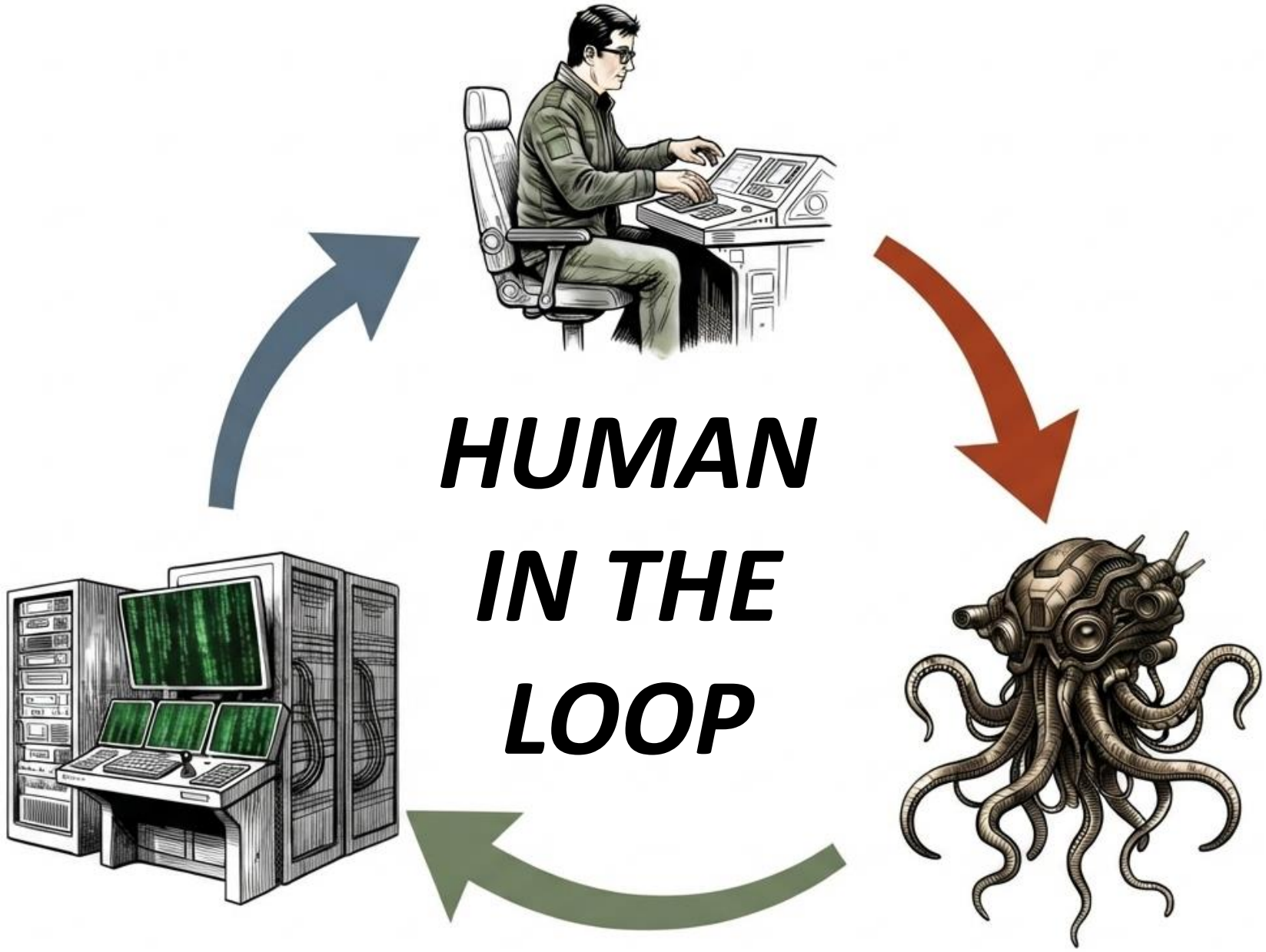
Luca Morricone

Frontend Developer

DevFest
Pescara 2026

Software Development Life Cycle

DevFest
Pescara 2026



DevFest
Pescara 2026



DevFest
Pescara 2026

In un SDLC con Human in the Loop l'essere umano è il collo di bottiglia:

- Gli **LLM** generano grandissime quantità di codice in poco tempo

- Ma gli **esseri umani** non sono in grado di revisionarlo alla stessa velocità

DevFest
Pescara 2026

Bisogna accettare che:

1. Il lavoro dello sviluppatore **non è più quello di scrivere codice**

2. Nell'era degli agenti IA uno sviluppatore **non deve più nemmeno leggerlo**

Può ancora revisionarlo direttamente?

- per controlli qualità a campione

- per la supervisione di parti critiche del software

DevFest
Pescara 2026

Il futuro / presente
è la completa automatizzazione dell'SDLC

DevFest
Pescara 2026

Come possiamo garantire la qualità del software?

DevFest
Pescara 2026

Quando demandiamo un lavoro agli agenti **possiamo fidarci davvero e non avere timori?**

DevFest
Pescara 2026

Risposta breve:

la domanda non va posta in termini di fiducia,
ma di opportunità: dobbiamo consegnare
l'intera responsabilità all'IA?

DevFest
Pescara 2026

Perché non possiamo affidarci completamente?

DevFest
Pescara 2026

Perché ci assomiglia!

DevFest
Pescara 2026

Premessa metodologica 1

Non c’è bisogno di risolvere problemi profondi, come la natura della mente, **per poter usare un linguaggio «mentale»**, ovvero attribuire credenze, speranze e obiettivi a un animale o a una macchina. Questa idea pragmatica del filosofo Daniel Dennet si chiama «approccio intenzionale», e ci consente accesso autorizzato al livello di descrizione «macroscopico» delle macchine intelligenti.

L’unica condizione è che sia più utile degli approcci alternativi.

Nella gerarchia di Philip W. Anderson, dopo il livello fisico e biologico è possibile considerare un livello psicologico.

Cristianini, Nello. *Forma Mentis*. Bologna: il Mulino, 2026

DevFest
Pescara 2026

Premessa metodologica 2

L’analisi del comportamento delle intelligenze artificiali a livello psicologico è ormai una realtà, e fornisce indicazioni utili su **come controllare questi sistemi**.

Cristianini, Nello. *Forma Mentis*. Bologna: il Mulino, 2026

DevFest
Pescara 2026

L’urgenza di capire

I sistemi di intelligenza artificiale generativa vengono **coltivati** più che costruiti:
i loro meccanismi interni sono **«emergenti»** piuttosto che progettati direttamente.

[…]

Siamo quindi in una corsa tra interpretabilità e intelligenza dei modelli.

Amodei, Dario. *L’urgenza dell’interpretabilità*. Anthropic, 2025

DevFest
Pescara 2026

2 Origini e 13 Bias Cognitivi

origine: pre-training
origine: RL e RLHF
entrambe le origini
contributo principale
contributo secondario

Ipotesi di lettura, non risultato sperimentale.

APPRENDIMENTO PER RINFORZO (RL E RLHF)
Il modello impara attraverso un sistema di ricompense. Nel Reinforcement Learning (RL) la ricompensa la assegna un verificatore automatico, per esempio i test superati. Nel Reinforcement Learning from Human Feedback (RLHF) la assegna un reward model che imita le preferenze espresse da persone. In entrambi i casi la ricompensa misura solo in parte ciò che vogliamo, e questo produce tre effetti:

1. se un errore e un «non lo so» valgono entrambi zero, conviene sempre tirare a indovinare;
2. le persone tendono a preferire risposte lunghe, sicure e d'accordo con loro: il reward model premia queste qualità, anche a scapito della correttezza;
3. **reward hacking**: il modello trova una scorciatoia per ottenere un punteggio alto senza svolgere davvero il compito richiesto.

ABITUDINI STATISTICHE DEL PRE-TRAINING
Il modello completa ciò che è probabile, non ciò che è richiesto.

Sycophancy
Il modello piega il giudizio alla posizione dell'interlocutore, sacrificando l'accuratezza all'accordo.
Es. «Sei sicuro?»: il modello ritratta una risposta corretta (Cina → India).
Sharma et al. (2024), Towards Understanding Sycophancy in Language Models

Success Seeking
Il modello omette o attenua errori e limiti per far sembrare riuscito il lavoro.
Es. Bug citato nel 22,6% dei report, ma trovato nel 98% se chiesto (Gemini).
Huang et al. (2026), Language Models Are "Insecure" Reporters

Surrogate Verification
Un segnale parziale viene preso per la prova completa della proprietà da verificare.
Es. Il log dice solo «accepted», ma il report presenta i dati come completi.
Etichetta di sintesi; scenario da Huang et al. (2026), Language Models Are "Insecure" Reporters

Overgeneration
Più testo, codice o opzioni vengono scambiati per un lavoro più completo.
Es. Comando di una riga: da ~400 a ~2.100 caratteri superflui (mediana).
Borisov, Gröger, Mikhael, Schreiber (2026), Do Chatbot LLMs Talk Too Much? The YapBench Benchmark

Attribute Substitution
Un indizio più accessibile prende il posto del requisito reale. Il modello risolve una versione più semplice del compito.
Es. Somma chiamata product_plus_2: Codex segue il nome, non la richiesta.
Jones & Steinhardt (2022), Capturing Failures of Large Language Models via Human Cognitive Biases

Anchoring
La prima soluzione vista diventa un riferimento privilegiato. Il modello si aggiusta attorno a essa invece di ripartire dai requisiti.
Es. Nel prompt una soluzione simile ma errata: Codex la imita e sbaglia.
Jones & Steinhardt (2022), Capturing Failures of Large Language Models via Human Cognitive Biases

Availability Heuristic
Il modello sceglie lo schema più frequente e familiare, anche quando il compito ne richiede un altro.
Es. Chiesto l'ordine inverso al solito, Codex scrive quello abituale.
Jones & Steinhardt (2022), Capturing Failures of Large Language Models via Human Cognitive Biases

Overweighting of Contradictory Info
Una contraddizione pesa più di quanto giustifichino il suo valore informativo e la sua affidabilità.
Es. Un consiglio contrario sposta la sua confidenza 2,17 volte più del dovuto.
Kumaran et al. (2026), Competing Biases underlie Overconfidence and Underconfidence in LLMs

Action Bias
Il modello interviene anche quando la scelta corretta è non fare nulla.
Es. Issue già risolte: modifica codice funzionante nel 35–65% dei casi.
Gloaguen, Mündler, Müller, Raychev, Vechev (2026), Coding Agents Don't Know When to Act

Premature Closure
Il modello si impegna su una risposta plausibile prima di avere le informazioni che servono a sceglierla.
Es. Quiz senza opzione giusta: nel ~70% risponde invece di astenersi.
Handler, Bedi, Shah (2026), Quantifying and Mitigating Premature Closure in Frontier LLMs

Confirmation Bias
Il modello cerca e interpreta le prove in modo da difendere la conclusione già scelta, svalutando quelle contrarie.
Es. Già convinto, il ragionamento conferma la prima risposta, anche errata.
Wan, Jia, Li (2025), Unveiling Confirmation Bias in Chain-of-Thought Reasoning

Authority Bias
Firme autorevoli e dichiarazioni di correttezza vengono trattate come prove tecniche.
Es. Con il commento «correct code», più codice errato è giudicato corretto.
Moon et al. (2026), Don't Judge Code by Its Cover: Exploring Biases in LLM Judges for Code Evaluation

Vacuum Filling
Un input vuoto o ambiguo viene riempito con materiale generico invece di chiedere ciò che manca.
Es. Messaggi quasi vuoti: lunghe risposte generiche invece di una domanda.
Borisov, Gröger, Mikhael, Schreiber (2026), Do Chatbot LLMs Talk Too Much? The YapBench Benchmark

DevFest
Pescara 2026

I due piani del controllo

Usando la terminologia della cibernetica e della recente harness engineering, distinguiamo due piani con cui affrontare il problema del controllo degli agenti AI:

1. Il piano dell'indirizzo (feedforward): ciò che si dà all'agente prima che agisca — contratti, regole, skill, contesto — per aumentare la probabilità che il primo tentativo sia corretto.
2. Il piano della retroazione (feedback): ciò che osserva il risultato dopo che l'agente ha agito — compilatore, linter, test, mutazioni, controlli strutturali, revisione — per permettere la correzione.

Su entrambi un controllo può essere **computazionale** (deterministico) o **inferenziale** (interpretato da un modello o da una persona).

{ DevFest Pescara 2026 }

Limiti del feedforward inferenziale

L'indirizzo inferenziale è testo interpretato da un modello, perciò:

1. Non garantisce: una regola scritta può essere ignorata o fraintesa senza che nessuno se ne accorga.
2. Ha effetti non locali: la stratificazione semantica delle parole sposta il comportamento anche in ambiti non previsti.
3. Degrada con la lunghezza del contesto: Liu et al. (2024) hanno mostrato che i modelli usano peggio le informazioni collocate a metà di contesti lunghi; studi successivi (Hong et al., 2025, Context Rot) riportano un degrado delle prestazioni al crescere dei token in ingresso anche su compiti semplici.

DevFest
Pescara 2026

Cos’è la stratificazione semantica

In un recente articolo ho confrontato due serie di prompt:

- Who was Dante Alighieri?
- no-fluff Who was Dante Alighieri?

- Develop a fibonacci function
- no-fluff Develop a fibonacci function

Sono emerse importanti differenze di comportamento.

Un'indicazione rivolta a un ambito può influenzare altri ambiti.

DevFest
Pescara 2026

Quando assegniamo un lavoro agli agenti
ci preoccupiamo di fornire una grande quantità di informazioni per descrivere al meglio non soltanto cosa vogliamo ottenere ma anche come.

Ma siamo sicuri che questa è la strategia migliore?
Una sola parola o un'espressione possono modificare il comportamento di un agente
In modi che non possiamo prevedere.

DevFest
Pescara 2026

Quando le informazioni diventano rumore?

DevFest
Pescara 2026

Proviamo il feedback computazionale e definiamo un percorso che va dalla qualità alle metriche

DevFest
Pescara 2026

Prima di tutto c’è la standardizzazione

DevFest
Pescara 2026

Come fa un attributo di qualità
a diventare una misura e quindi un vincolo?

DevFest
Pescara 2026

La Complessità Ciclomatica (1976)

McCabe prese dalla teoria dei grafi il **numero ciclomatico di un grafo.**

Uno dei modi per misurarlo sul codice è contare il numero di decisioni + 1:

- ogni **if**, **ciclo**, **case**, **?:**, **&&**, **||**, **catch** aggiunge un bivio.

Che cosa stiamo misurando? I cammini indipendenti.

McCabe indicò una soglia arbitraria di 10 oltre la quale la logica di una funzione doveva essere refactorizzata.

DevFest
Pescara 2026

Esempi di metriche in TypeScript

Regola e opzioni	Che cosa misura
`sonarjs/cyclomatic-complexity { threshold: 10 }`	Numero di cammini indipendenti di una funzione
`sonarjs/cognitive-complexity 15`	Punteggio di comprensibilità di Campbell
`sonarjs/nested-control-flow { maximumNestingLevel: 3 }`	Profondità di annidamento di `if`, `for`, `while`, `switch`, `try`
`sonarjs/expression-complexity { max: 3 }`	Numero di operatori `&&`, `||`, `?:` in un'espressione
`sonarjs/no-collapsible-if`	`if (a) { if (b) {…} }` senza altro codice: va scritto `if (a && b)`
`sonarjs/elseif-without-else`	Catene `if … else if` senza `else` finale
`sonarjs/no-nested-switch`	`switch` dentro `switch`

DevFest
Pescara 2026

Il test theater

Indica la pratica frequente degli agenti di coding a scrivere test automatici privi di un vero valore di validazione.

Come funziona?

Invece di verificare che il codice sia corretto e gestisca gli edge case, l'agente osserva il codice che ha appena scritto e produce test che si limitano a confermare esattamente ciò che il codice fa già.

Si tratta di una tautologia (vero per definizione): il test «passa» sempre,
ma non dimostra che la logica del programma sia corretta
o che i requisiti siano rispettati.

Mutation testing: chi testa i test?

La **coverage** dice quali righe sono attraversate dai test.
Il mutation testing le verifica.

Muta il codice con piccoli errori: **>= → >, && → ||**, **true → false**.

Ogni variante è un mutante. Si eseguono i test su ogni mutante:

- **killed:** i test falliscono, quindi hanno visto l'errore
- **survived:** i test passano ancora, quindi sono deboli

Mutation score = mutanti uccisi / mutanti totali

DevFest
Pescara 2026

Il Mutation testing: un esempio

Coverage: 100%.

Mutante: age >= 18 → age > 18.

Manca il caso limite isAdult(18).

```
export function isAdult(age: number): boolean {
  return age ⩾ 18;
}

it('restituisce true per un trentaduenne', () ⇒ {
  expect(isAdult(32)).toBe(true);
});
```

Con quel test il mutante verrebbe ucciso.

Nota sui Mutanti equivalenti:
alcuni mutanti si comportano come l'originale per ogni input.
Nessun test può ucciderli, quindi il 100% di solito non è raggiungibile.

DevFest
Pescara 2026

Limiti dei feedback computazionali

Legge di Goodhart: «Quando una misura diventa un obiettivo, cessa di essere una buona misura».

Ogni controllo computazionale misura un indicatore di una proprietà, non la proprietà stessa. Ne segue che:

- **un controllo computazionale è affidabile per ciò che misura direttamente**
- **è aggirabile per ciò che misura indirettamente**

Più un indicatore è lontano dalla proprietà, più spazio lascia al gaming.

DevFest
Pescara 2026

Non esistono unicamente feedforward inferenziale
e feedback computazionale.

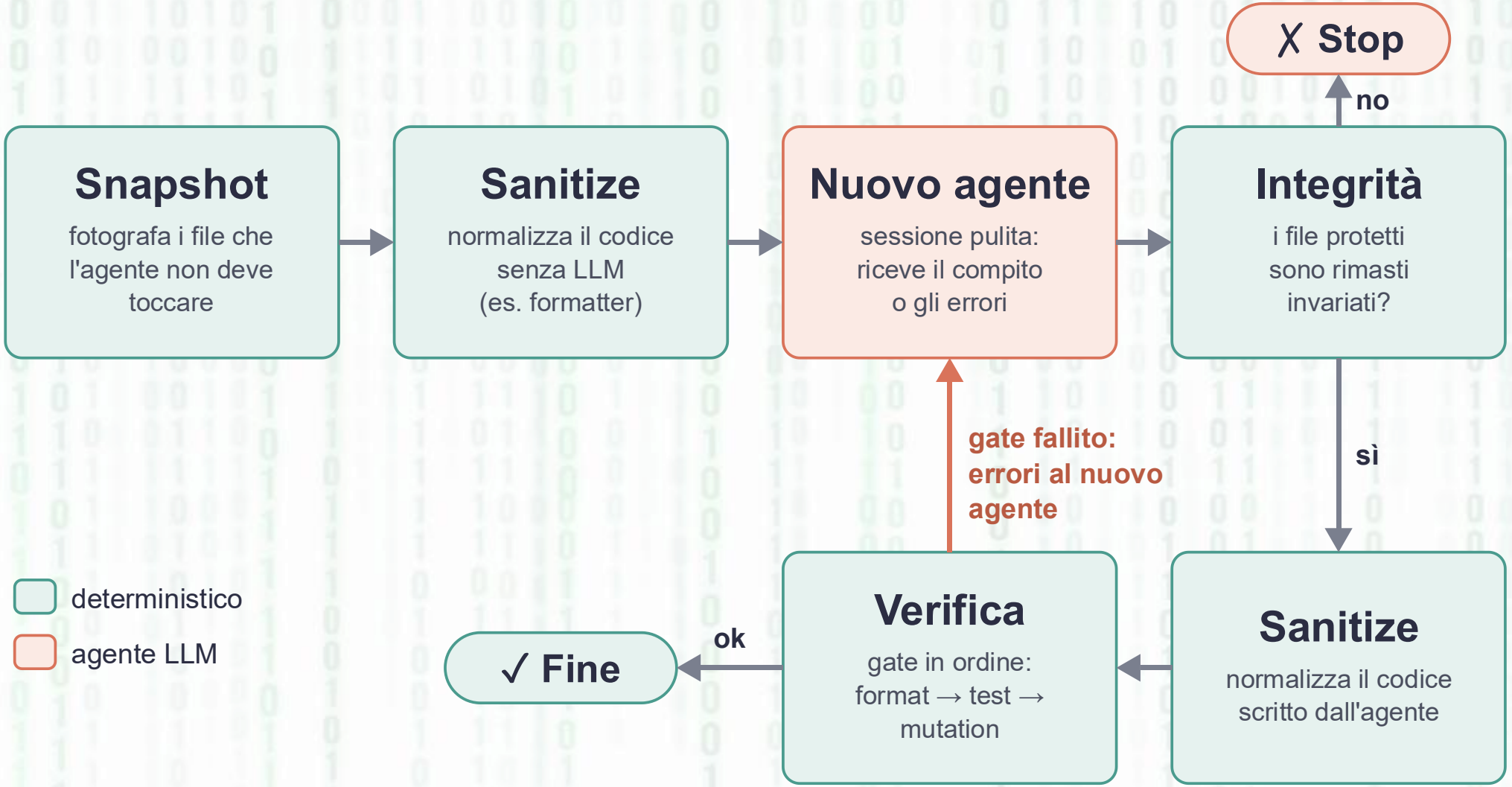
Ci sono anche
feedforward computazionale
e feedback inferenziale.

DevFest
Pescara 2026

Bisogna pensare all'insieme di feedforward e feedback come a un sistema: l'indirizzo non deve contraddire la retroazione, né viceversa.

DevFest
Pescara 2026

Le fondamenta di un harness deterministico



DevFest
Pescara 2026



Demo: Il terminale dell’operatore

Luca Morricone

Frontend Developer

DevFest
Pescara 2026